THE LEDGER FRAMEWORK

Manifesto and Architectural Constitution

A Unified, Closed-System, Event-Driven Post-Trade Architecture

R. Delloye — 13 July 2026 — Version 1.2

Abstract

Reconciliation exists because many systems store the same facts, and independently maintained copies eventually disagree. The Ledger removes the cause: every fact is recorded exactly once, in an immutable log of atomic transactions, and everything else — balances, valuations, profit and loss, reports — is a deterministic projection of that single record, reproducible by any party and reconstructible at any past date; illegal states are not checked for but made unrepresentable. This manifesto is the project's founding document: assuming no prior knowledge, it states the objective, the eight commitments that govern every design decision, the objects the system manipulates, the arrows between them, and the machines that evaluate those arrows — the architecture, not the engineering, with implementation questions named and left to the specification. It closes with the minimum requirements for the Event Monitor, the Events Executor, and the pricing layers, and with the rule that governs everything after: all future specifications and implementations are rated against this document, never the other way round.

Margin identifiers C-sec.n address the normative clauses. The numbering is append-only: identifiers are never reused and never renumbered; an amendment changes a clause's text or appends a new one.

---

1. The Objective and the Problem

The objective of the Ledger project is to build a clean post-trade and risk-management platform: C-1.1
one system of record from which positions, valuations, profit and loss, collateral, and reports are all derived — exactly, deterministically, and reproducibly.

The obstacle is the architecture of today's infrastructure. Post-trade processing is not one system but many — trading systems, risk engines, general ledgers, custodians, regulatory engines — and each holds its own copy of positions, cash balances, corporate-action adjustments, and lifecycle states, updated on its own schedule by its own logic. The copies diverge, and reconciliation — detecting and repairing the divergences — is one of the largest operational costs and risks in the industry. The root cause is architectural: **the same fact is stored more than once.** Two C-1.2
stores of one fact maintained by independent logic will eventually disagree, and no amount of C-1.3
process removes a failure mode the architecture itself provides.

The Ledger removes it at the root. There is exactly one canonical record — an ordered, append- C-1.4
only stream of atomic transactions — and every other number is computed on demand as a projection of that stream: balances, valuations, profit-and-loss statements, balance sheets, regulatory reports. Two projections of one record cannot disagree about any fact the record C-1.5
contains. That sentence is the design's entire ambition; the rest of this document is what it forces.

2. The Eight Commitments

Eight commitments are non-negotiable, and every design decision in this project is tested against them.

Full auditability. Every number is traceable, mechanically and completely, to the recorded events that produced it. History is never rewritten: a mistake is repaired by a recorded correction that names what it supersedes, so the error and its repair both survive. C-2.1

Full reproducibility. Given the same record, any party — the firm, a counterparty, an auditor, a regulator — recomputes the same state, the same views, the same contract outputs, to the last minor unit. C-2.2

Time travel embedded in the design. The exact state of the system at any past moment is reconstructible from the record alone, at any time, forever — independently of whether the software that produced the events still exists. C-2.3

Correctness. Illegal states are made unrepresentable wherever possible: a check can be forgotten or bypassed; a type cannot. Where wrongness cannot be prevented — a contract can always compute the wrong economics — it is made mechanically detectable and deterministically repairable. Defaults fail closed: the system stops rather than improvises. C-2.4

Testability. The ledger is correct because it is built to be tested. Every step is a pure, versioned function with explicit inputs, explicit outputs, and strong types — typing alone eliminates whole classes of defects before a single test runs — and every core invariant is expressed as an executable check over generated products, events, and histories, never defended only in prose. Correctness then composes: because the steps are pure and their composition is fixed — the map, then the fold — verifying each part verifies the whole. A property that cannot be tested mechanically is redesigned until it can be. This is not an afterthought: the architecture is deliberately shaped so that its core guarantees are executable — property-based testing is a construction principle, not a phase at the end. C-2.5

Clarity. One name per component and one component per name; behaviour carried as declared data rather than hidden in code; every claim stated so that it can be checked by a reader with no prior exposure. C-2.6

Order. Events do not commute: each event's transaction is computed against the ledger all earlier events have built, so the same events in a different order are a different history — buy shares then meet the dividend's record date and you are entitled; meet the record date then buy and you are not. Map and fold therefore advance one event at a time — map one event, fold its transaction, only then map the next — and the log's total order is part of the meaning of the record, never bookkeeping. This commitment forces the single writer, the single total order, and the refusal to reorder anything unless harmlessness is proved. C-2.7

Simulability. The current state is the initial state plus the fold of all transactions; the events that produced it are one path among the paths that could have occurred. The machinery must therefore run unchanged on hypothetical event streams: simulated paths branched from any recorded state, driven by generated events, folded by the same contracts and the same door — for pricing, stress testing, and testing the implementation across thousands of paths. The one non-record input of a simulation is the seed, recorded so that every path replays exactly. This commitment forces contract purity, the confinement of the clock, and the wall that keeps effects — settlement, models — outside the fold. We live in a simulation: production is simply the one path whose events happened to be real. C-2.8

The document builds in a fixed order: the whole architecture in one picture (Section 3); the objects it manipulates (Section 4); the machines that run it (Section 5); the smart contracts that

animate it (Section 6); and then the consequences, domain by domain, ending with minimum requirements and scope.

3. The Whole Architecture in One Picture

Everything this framework does can be stated as a map followed by a fold — two words borrowed from programming, worth a line each. In plain words: transform each event, then accumulate the results. A *map* processes a list element by element: given a way to turn an a into a b, it carries [a] to [b], one element at a time. A *fold* accumulates a list into a single result: each element of [a] is processed in turn and absorbed into a running value, [a] to b. Four concepts complete the picture, each defined precisely in Section 4: a **unit** is anything that can be held or owed — a stock, a bond, an option, a currency; a **wallet** is where units are held, in signed quantities called balances; a **transaction** is the atomic bundle of changes the ledger records; and a **projection** is any view computed from the record. Everything else this section names, it explains as it goes. C-3.1 C-3.2

The world presents the system with a list of events, E = [e]. Some arrive from outside: trades executed in the market, corporate-action notices, fixings and other observations, captured and recorded at the boundary. Others arise from the instruments themselves — a coupon date arriving, an expiry falling due, a close printing below a declared barrier — and these are not received but noticed: every unit declares, from its terms, the conditions its life requires watching, and the Event Monitor evaluates those watches against the clock and the recorded observations, emitting an event onto the record the moment a condition is met. C-3.3

Each event is mapped into a transaction — itself a bundle of moves, state changes, and terms changes, defined precisely in the next section — and the responsibility for the map divides in two. The smart contracts are the mapping functions: each is a stateless piece of code that, given one event and the current state of the ledger, computes the transaction the event requires. The Events Executor applies the map over the list, E = [e] → T = [tx]: for each event it fires the responsible contract at the right time, hands it the event together with the current projection of the ledger, and collects the proposed transaction. Many contracts, one Events Executor; the contracts carry all the product knowledge, the Events Executor carries none. C-3.4

The resulting list of transactions is then folded — reduced, one transaction at a time — into the current state of the system: balances, unit states, and product terms. The Transaction Executor performs the fold, T = [tx] → Ledger: it serialises, validates, and applies each transaction, and the immutable log is precisely the list being folded. Every view — a balance, a valuation, a report — is a further fold of the same list. C-3.5

Three kinds of thing appear in this picture, and they must never be confused: data, functions, and machines. The data are Event, Transaction, and Ledger. The functions are the arrows between the data. The machines are what evaluate the arrows — one machine per arrow. Typed: C-3.6

```
watch    : (Record, Clock)       -> [Event]      -- the Event Monitor
contract : (Event, Ledger)       -> Transaction  -- the Events Executor
apply    : (Transaction, Ledger) -> Ledger       -- the Transaction Executor

step (ledger, e) = apply (contract (e, ledger), ledger)  -- map, then fold
final ledger     = fold step over E = [e]
```

One instrument can walk the whole pipeline. Take a one-touch option that pays 1 USD if the spot of its underlying ever trades at or below a fixed barrier. At registration, its terms declare the watch: the underlying's recorded prices against the barrier. Then, the day a low print is recorded:

- **watch** — the condition is met: the Event Monitor emits the touch event onto the record.

- **contract** — the Events Executor fires the option's contract, handing it the event and the current projection of the ledger; the contract reads the terms and who holds the option, and returns one transaction: a single move of 1 USD from the issuer's wallet to the holder's wallet, plus one state change on the unit — *triggered*.
- **apply** — the Transaction Executor admits it, and the fold advances: the ledger now shows the cash with the holder and the option triggered.

Should the Monitor ever emit the touch again, the next firing finds the unit already triggered and proposes nothing — the recorded state is exactly what makes a duplicate harmless.

Time deserves separate attention. The clock is the single input that is not on the record, and the Event Monitor is the single place it is consulted. The event the Monitor emits is recorded, so everything downstream of it is a function of the record alone — which is exactly why a late firing produces the identical transaction. Because watches are declared and the Monitor is deterministic, even the emission of events is verifiable: given the record and the clock, anyone can recompute which events should have been emitted, and when. C-3.7

Both contract and apply can refuse — a contract may find nothing due; apply rejects a malformed transaction and writes nothing — and a refusal is a returned value, never a half-applied state. C-3.8

Two remarks complete the picture. First, the map is not blind to state: to turn a dividend event into cash moves, the contract must read who owns what at that moment — the fold of everything admitted so far. Map and fold therefore advance together, one event at a time, in a single total order (Section 5). Second, the map is proposed while the fold is authoritative: a transaction joins the list only when the Transaction Executor admits it, and economic verification (Section 13) is simply re-running the map on the recorded event and comparing — which is why both the events and the transactions are on the record. C-3.9 C-3.10

A note for readers who know Event Sourcing, because the resemblance is real and the difference matters. The Ledger is event-driven because finance is: trades execute, prices print, coupons mature, and barriers are touched whether or not the software cares. Events are recorded because reality presents them — not because event sourcing was chosen as a pattern. The architectural difference is where authority sits. In event sourcing, the event stream is the canonical history, state is whatever the current code derives from it, and reinterpreting history through new code is a feature. Here, events are inputs to computation, never the accounting record: contracts transform them into proposed transactions, the Transaction Executor admits or refuses — an admission door event sourcing does not have — and the admitted transaction log alone is the canonical history. Replay re-reads transactions and never re-executes contracts, so history cannot be silently reinterpreted; a wrong interpretation is repaired by a compensating transaction, in the open. The events stay on the record for one purpose: so the map can be re-run and verified. Verification lives with the events; authority lives with the transactions. C-3.11

4. The Objects

At the highest level, a ledger is three things and a record: units — what can be held; wallets — where it is held; balances — how much of each unit each wallet holds; and the immutable log that records every change to them. This section defines each object once; nothing later in the document introduces another. C-4.1

A **unit** is anything that can be held or owed inside the ledger: a currency (USD in cents), an equity security (identified by its ISIN), a bond, a listed or OTC derivative, a securities loan, a managed-account mandate, a margin obligation under a collateral agreement (a CSA), or any other contractual instrument. Units are first-class and extensible: adding a new instrument type adds a new element to the universe of units, never a modification of the machinery. Because C-4.2

every unit — asset, liability, or contractual obligation — is represented uniformly, the same move machinery and the same conservation discipline apply without special cases.

Units divide into valued and non-valued. A unit is **valued** if and only if it represents an economic claim whose value exists independently of the governance or continued operation of the instrument that created it. A stock, a bond, and a client's redemption claim on a managed account are valued. A mandate's rulebook, a portfolio-swap reset mechanism, or a quantitative investment strategy's governance shell is non-valued: its price is undefined — not zero — because pricing the governance structure beside the claim it governs would count the same economic reality twice. C-4.3

Every unit carries three things: its terms (the contractual text, versioned and append-only), its state (where it stands in its lifecycle), and its smart contracts — the executable form of those terms. From its terms follow its **watches**: the dates, observations, and conditions the Event Monitor must observe on its behalf. Declaring them is the unit's duty — at registration, and again whenever a later transaction creates a new obligation — and the declaration is exhaustive: the Monitor does exactly what the units have asked of it, nothing more and nothing less. C-4.4

A **wallet** is a named logical partition of position space — a container, and deliberately nothing more. Its state at any moment is a signed quantity, its balance, for each unit it holds. A wallet names two parties: a **manager**, who is authorised to perform transactions on it, and a **beneficial owner**, who is legally accountable for the book. Wallets carry no smart contract and no logic of their own; where a wallet appears to need behaviour — the rules of a managed account, for instance — that behaviour is a unit (the mandate). C-4.5

An **atomic move** is the sole operation that changes a balance: it transfers a positive quantity of exactly one unit from a source wallet to a destination wallet in a single indivisible step. Quantities are exact integers in minor units; where an event divides unevenly, the rounding convention is part of the declared terms and the remainder is booked explicitly as its own move — nothing is ever silently dropped. Because a move only transfers, it can never create or destroy quantity. C-4.6

A **transaction** is an atomic, all-or-nothing bundle of four kinds of change: moves, unit-state changes, position-state changes, and terms changes. The four align one-for-one with the four components of the ledger's state: balances, and the three homes of Section 7. A transaction applies in full or not at all. Some transactions carry no moves (for example, the registration of a new unit or the recording of a barrier breach). C-4.7

The **immutable log** is the ordered, append-only, hash-chained record of every admitted transaction — hash-chained so that any alteration of history is detectable. It is the sole canonical source of truth. Three words are used precisely throughout this document: the **log** is the recorded sequence itself; the **ledger** is the state obtained by folding the log; and the **record** is everything the log carries. Every observation whose reproduction the framework guarantees enters the record only as a moveless transaction — admitted through the one Transaction Executor and folded into a home — the *observation-recording transaction.* An event is a trigger: it carries timing, not admitted data. No observation is exempt from the one door, so the single canonical record of Section 1 and the single writer of Section 5 extend without exception to every observation. In the typed picture of Section 3 the watch reads the Record while the contract and apply consume the Ledger, unchanged, because the observations a contract reads are home facts already folded into it. "On the record" means present there, and a projection is any view computed from it. C-4.8

Every counterparty outside the system is represented by a virtual wallet inside it, so the system is closed. Conservation holds by construction, not by check. C-4.9

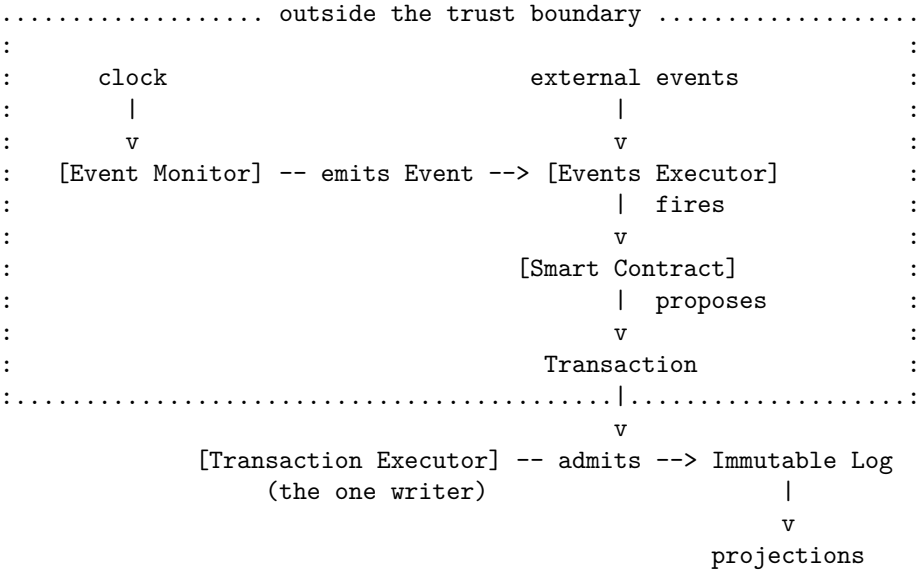
When a balance must carry more information than a single number, it generalises to a signed vector. The first coordinate is ownership, and only the owned coordinate carries economic value and drives profit and loss. Every further coordinate records an arrangement in which possession or C-4.10

use moves without ownership — collateral, lending, and their kin — and carries mass, never value. How many further coordinates exist, and how each is indexed, is decided in the specification, on the specialists' advice, under three tests this constitution fixes: a coordinate is admitted only where a distinct real-world action can change it independently of ownership; the grain is fine enough that two distinct real- world facts can never net to nothing — an encumbrance is never hidden by an offsetting one; and conservation holds at the finest declared grain. A non-owned C-4.11
coordinate records possession and encumbrance under a named agreement: no quantity reaches it without reference to the agreement unit that governs it, and a lifecycle event extinguishes value, never mass — a unit leaves the ledger only from the zero vector. Anything computable from the coordinates is a projection, computed when needed and never stored.

The kinds of event the system can process are finite, closed, and declared on the record before C-4.12
any event of a kind crosses; registration names the responsible routing, so an event of a declared kind always has its contract. An arrival the system cannot process — an undeclared kind, an unresolvable reference, a payload that fails validation — is never guessed at and never lost: it is captured and recorded through the one door as a moveless fact carrying the provenance of its arrival, and it stands as a visible open item until its kind is registered or a person disposes of it. The guarantee runs from arrival: whether the world's events reach the boundary at all is a perimeter matter, named as a trust assumption and reconciled against external authorities, never assumed. The mechanics of the registry, the routing, and the quarantine are the specification's.

5. The Machines

The framework has three machines — one for each arrow of the picture in Section 3. All three C-5.1
are generic machinery carrying no product knowledge, and only the last ever writes the ledger: three roles, one writer. In one picture — machines in brackets, data on the arrows, the dotted line the trust boundary:



The **Event Monitor** owns time and attention. It holds every declared watch and evaluates C-5.2
them against the clock and the recorded observations, emitting an event onto the record the moment a condition is met. It holds no private list — the watches it evaluates are on the record — and it emits events, never transactions. The relationship between the units and the Monitor is a recorded handshake. Each declared watch is acknowledged by the Monitor, and the acknowledgement — an event like any other, flowing through the same pipeline — is written into the unit's state; a watch is thus visibly *declared*, then *armed*, then *fired* or *expired*.

A declared watch that is never armed is therefore a visible gap, never a silent one. The full chain is on the record and can be walked in either direction: register a unit → the conditions it declares → the acknowledgements → the events → the transactions → the state updates. The Monitor can affect timeliness and liveness, but it cannot affect the correctness of ledger state, because it only emits events — it never proposes transactions. C-5.3

The **Events Executor** owns delivery. It captures and records the events arriving from outside, and, for each event — arrived or emitted — it fires the responsible smart contract, handing it the event together with the current projection of the ledger, and collects the proposed transaction. It runs the contracts; it never writes the ledger; its output is only proposals. C-5.4

The **Transaction Executor** owns the record. It is the gatekeeper of the ledger door: the single component through which every transaction must pass, and the only component permitted to change the ledger's state. It validates structure — conservation is automatic; authorisation, idempotence, consistency of reference, and writer discipline are enforced — and, if the transaction is admissible, appends it to the immutable log. Every proposed transaction carries an identifier derived from its cause, so a retried or duplicated proposal is recognised and committed only once. It creates nothing and runs nothing; it admits or refuses. There is no other way to change the ledger's state. C-5.5 C-5.6

The one-touch option of Section 3 met all three machines in order: the Monitor noticed the low print and emitted the touch; the Events Executor fired the option's contract with the event and the projection; the Transaction Executor admitted the single move and the *triggered* state. One event, three machines, one write.

The single-writer discipline is a correctness requirement. The Transaction Executor therefore presents to the log a strictly monotonic sequence of committed transactions; where two simultaneous events on the same instrument do not commute and declare no precedence, it refuses rather than guesses an order. C-5.7

The roles sit at different levels of trust. The Transaction Executor is the trusted component: the ledger's integrity rests on it and on nothing else. The contracts, the Event Monitor, and the Events Executor are outside the trust boundary, untrusted in different ways: a contract can be economically wrong, so its output is verified by recomputation (Section 13); the Monitor and the Events Executor cannot corrupt the ledger at all — a late emission or firing produces the identical transaction, a duplicate is committed once, a spurious one finds nothing due and is refused — but the timeliness of the system depends on them. Integrity owes them nothing; liveness is conditional on them. Section 14 states their minimum requirements. C-5.8

6. Smart Contracts: Event-Driven, Pure, and Faithful to the Legal Contract

Financial instruments are not passive data records; they are active contracts that, when triggered by an event, emit the transactions reflecting the economic consequences of that event. The Ledger performs none of this product-specific logic: it admits well-formed transactions and records them. The logic that decides what a dividend, an expiry, a barrier knock, or a corporate action means for a particular instrument resides in the instrument's smart contract, outside the ledger's trust boundary. Since the ledger is only a transaction processor, anything coming from outside must be converted into a transaction, and the smart contract is the converter. C-6.1

Examples illustrate the pattern:

- A dividend is announced by the issuer, and the announcement is recorded when received. The announcement fires the stock's smart contract once — not to pay, but to register the

watches the dividend creates: its record date and its payment date. When the record date arrives, the Event Monitor emits the event and the Events Executor fires the contract again; it reads the positions as the ledger then stands and records each holder's entitlement — a moveless transaction: no cash changes hands, but the position state of every holder is updated with the entitlement it is owed. When the payment date arrives, the Monitor emits again and the contract fires a third time, emitting the cash moves against the recorded entitlements. **Three firings, three transactions, one stateless contract.**

- A cash-settled future declares, from its terms, the watches its life requires: each day's settlement-price publication and its expiry date. On each recorded settlement print the contract fires and emits the day's variation-margin moves, updating in the same transaction the position state that records the margin applied and the settlement level used; on expiry the Monitor emits the final event, and the contract unwinds the position and pays the last cash settlement — one atomic transaction.
- A listed option's smart contract is triggered at expiry by the official settlement price; it evaluates whether the option is in-the-money and emits the transaction that extinguishes the position and delivers the underlying or the cash amount, in one atomic step.
- A note's barrier is a declared watch on the underlying's official close. The close is observed and recorded; the Event Monitor emits the knock event, and the note's contract fires and records the breach — a lifecycle state change that may involve no moves at all.

Two principles govern all of these. First, everything a smart contract consumes must itself be on the record. Second, a contract never waits and never remembers: the watch waits in the record, the Event Monitor notices when its condition is met, the Events Executor delivers the firing — and whatever a contract must carry from one firing to the next, it records. C-6.2 C-6.3

Purity and statelessness follow from the same demand. Every smart contract is a pure, deterministic function of its inputs: the declared terms of the instrument, the recorded observations, and the visible unit and position state at the moment of invocation. Each contract is therefore a stateless, machine-readable encoding of the legal terms and conditions of the instrument it governs. C-6.4

The alignment with the legal contract runs deeper: every right and obligation the legal contract creates is represented in the ledger as a unit. A subscription is booked as an exchange — cash against the client's redemption claim — never as a transfer for nothing. Treating the client's redemption claim as a valued unit has direct consequences for fee accrual and NAV attribution in managed accounts; these consequences are resolved in the detailed specification consistently with the principles stated here. C-6.5 C-6.6

7. State: The Three Homes

Every unit progresses through a well-defined sequence of lifecycle states. Some elements of state are properties of the unit alone; others are joint properties of the unit and a specific wallet. C-7.1

The three-home state model encodes the distinction: (i) ProductTerms — the immutable, append-only versions of an instrument's contractual terms; (ii) UnitStatus — a materialised, replayable projection of events that affect the unit as a whole; and (iii) PositionState — per-wallet, per-unit attributes. Because all three are derived from the immutable log rather than being independent mutable stores, they can be discarded and rebuilt by replay at any time without loss of information. Each non-balance fact has exactly one legal writer. C-7.2 C-7.3

The three homes are not bookkeeping conveniences: together with the balances, they must carry **all the information the smart contracts require in order to execute** — the past fixings C-7.4

a variance swap has accrued, the last margin call a future has applied, the coupons a bond has already paid. Each such fact is written into its home by an earlier firing, so that the next firing finds its entire context on the record; if a contract would ever need to remember something, that something has a home.

The daily life of a cash-settled future shows all three homes at work. The official settlement price, once published and recorded, is the same fact for every holder: it lives in UnitStatus. The variation margin a particular holder paid or received yesterday depends on that holder's position: it lives in PositionState. And when the underlying stock splits, the contract's strike and multiplier are adjusted by appending a new terms version: that is a change to ProductTerms. One event — a settlement print, a margin payment, a split — lands in exactly one home, and C-7.5
the home is determined by what the fact is a property of: the unit alone, the unit-and-holder pair, or the contract's terms.

8. Valuation and Profit and Loss

Net Asset Value has an operational meaning before it has a formula. The initial NAV of a wallet C-8.1
is the cash it took to build the portfolio at inception; the current NAV is the cash you would hold if you unwound every position at prevailing prices; and the profit and loss (PnL) is the difference between the two.

At any point in time the NAV of a wallet is obtained by (i) replaying the transaction log to C-8.2
recover the wallet's current composition and (ii) multiplying that composition by the current price vector supplied by the pricing data layer, the sum running over the valued units only.

Because unit state affects value, the pricing function is also parameterised by the current lifecycle C-8.3
state of the instrument — obtained from the ledger, never from a store of its own.

A central and non-negotiable property follows: the profit and loss of any wallet is exactly the C-8.4
variation in its NAV. PnL between two dates is path-independent, and no second copy of "realised PnL" or "book cost" is ever maintained alongside the position record.

For the variation in NAV to be exactly profit, deposits and contributions must not create profit. C-8.5
Every inflow of value is one of three things: an exchange paired with an equal-valued outflow C-8.6
— settled-to-market margin is this case, a settlement extinguishing the day's exposure with no return obligation; a contribution or financing received against an obligation created in the same C-8.7
transaction and valued at fair value, any day-one difference between the inflow amount and the obligation's fair value being recognised as financing basis — collateral received under title transfer, cash included, is this case, owned by the receiver against an equivalent- return obligation that is itself a unit; or value held in custody without being owned, recorded on the collateral-received C-8.8
coordinate — collateral received under a security interest without right of use, segregated cash included, is this case. Where an inflow moves under a margin or collateral agreement, which case C-8.9
governs it is declared once, on that agreement unit. A deposit therefore cannot move profit and loss — not because a formula subtracts it, but because it cannot change net owned value.

9. Corporate Actions and the Market Data Operator

A corporate action is, first of all, a transaction like any other: a list of moves, together with C-9.1
unit-state changes and a new version of the instrument's terms, all applied atomically through the same Transaction Executor.

A corporate action also changes the meaning of all the market data recorded before it. To C-9.2
carry pre-event market data into the post-event frame, a transformation must be applied. The framework gives it a name: the **market data operator**. Every kind of market data is equipped

with its own operators, one per kind of event — declared once, when the kind of data is introduced and the corporate action is recorded, never improvised at the moment of reading. A split shows the variety: the spot scales; cash-per-share dividend figures scale while proportional (percentage) figures stand; index compositions are updated — in every case until fresh post-event data are recorded.

Three principles govern the operators. First, the operator is intrinsic to the pairing of the kind of data and the kind of event. Second, the ledger is the authority; the pricing data layer applies. Third, originals are never overwritten. The observed price stays on the record exactly as observed; the adjusted value is computed at each read. Derived quantities are recomputed from operator-adjusted inputs. C-9.3

10. Strategies, Virtual Ledgers, and Total Return Swaps

Nothing in this architecture is specific to real holdings. A **virtual ledger** is a complete instance of the same machine — units, wallets, balances, an immutable log, the three machines — whose holdings are notional: it settles nothing, and none of its wallets ever reaches a settlement instruction or the balance sheet. Virtual ledgers are subject to the same eight commitments as true ledgers, especially full reproducibility, time travel, and simulability. The word virtual keeps one meaning: on the far side of the settlement boundary. C-10.1

The motivating case is a quantitative investment strategy. A deterministic strategy is a smart contract whose rulebook is the terms of a (non-valued) strategy unit. The hypothetical portfolio it manages is a wallet in a virtual ledger. Each rebalancing is a recorded transaction triggered by stamped market observations. The strategy's level is that wallet's NAV — a projection computed by the same fold as any other; because the strategy ledger is replayable, any party holding the recorded rules and observations recomputes the level exactly, and an index restatement is a compensating entry, never a rewrite. C-10.2

Two disciplines make this safe. First, the bridge between ledgers is an observation, never a move. Second, the wall between models and the ledger holds level by level: the true ledger never runs the strategy; it consumes the level as an observation. C-10.3

When the strategy's trades are real, the strategy's output toward the world is an order (an instruction, not a fact). The executions that come back are external events, mapped by contracts and folded into a real wallet of the true ledger. The virtual ledger becomes the benchmark: it holds what the rules say the portfolio should be, the real wallet holds what execution achieved, and the difference between the two folds is the slippage — exact, never estimated. C-10.4

A total return swap is then one definition: the reference leg of a TRS is the NAV of a designated wallet (or virtual ledger). In every case the TRS is an ordinary unit in the true ledger whose smart contract, at each reset, reads the stamped NAV observation and emits the reset and financing moves. C-10.5

11. Structural Invariants and the Impossibility of Broken States

The Ledger is deliberately engineered so that certain classes of inconsistent or illegal state cannot be represented at all: C-11.1

- **Atomicity.** A transaction is applied in full or not at all. C-11.2
- **Consistency of reference.** A quantity and a price may be combined only when they refer to the same state of the instrument. After a two-for-one split, 1,000 shares at 100 become 2,000 at 50; multiplying the new count of 2,000 by the stale price of 100 would book a phantom valuation. C-11.3

- **Writer discipline.** Lifecycle facts are written only by the single legal writer for that fact. C-11.4

These three join the invariants established earlier — conservation by construction (Section 4), C-11.5
the append-only log (Section 4), the single writer (Section 5), the determinism of every arrow (Section 3), and idempotence under the cause-derived identifier (Sections 5 and 12) — to form the catalogue that the property-based tests of the Testability commitment exercise. Together they render the classes of internal reconciliation failure that dominate conventional architectures unreachable within the Ledger's scope: what cannot be represented cannot break.

12. Time Travel, Replayability, and Idempotence

Every historical state of the Ledger is exactly reconstructible by replaying the immutable C-12.1
transaction log. The system's standing duty is to compute both honest answers at any moment: what the book said then — the view as it was known, never rewritten by later arrivals — and what the numbers are with a later correction in force. Both are deterministic replays of the one log; how the two views are indexed is decided in the specification. Because replay reads C-12.2
only committed transactions and never re-executes smart contracts, the historical record is independent of the machinery that produced it.

Smart contracts must be idempotent. The mechanism is the cause-derived identifier of Section 5: C-12.3
a retried or late-arriving proposal for an already-processed event produces no duplicate effect.

A recorded event found wrong — a wrong price, a wrong payoff — is repaired forward, never C-12.4
edited. The discovery is itself a new recorded event that names the wrong one; the discrepancy stands as a visible open item; and, because money already moved cannot be unsent, the repairing compensating transactions are agreed with the counterparty and authorised by a person before they fire. Views recompute automatically; money never does. In phase 0 no correction fires without human intervention.

Where the record itself is corrupted by systemic failure, the remedy is the authorised fork: replay C-12.5
to a chosen point and rebuild forward on a new lineage that names its parent point — the rollback of a bad delivery, not a rewrite of history. The abandoned lineage is retained, never erased.

13. Two Layers of Correctness

The correctness of the Ledger rests on two distinct layers:

- **Structural correctness** is guaranteed by the Transaction Executor. These properties C-13.1
hold regardless of whether the smart contract that produced the transaction was correct, careless, or malicious.
- **Economic correctness** is not guaranteed at admission; it is made checkable by recompu- C-13.2
tation. A smart contract may emit a perfectly well-formed transaction that nevertheless books the wrong economics. The discrepancy is detected after the fact by recomputing the transaction from the recorded inputs. The difference constitutes a defect of the contract, not of the ledger, and it is repaired by a subsequent compensating transaction through the same door.

This separation is fundamental. The Ledger relies on structural correctness because it is C-13.3
guaranteed by construction; it expects economic correctness of its contracts and renders it mechanically verifiable. Detection is after the fact. The framework prefers a detectable, repairable error over a pretence of prevention it cannot deliver.

14. Minimum Requirements

14.1 The Event Monitor and the Events Executor

The Event Monitor and the Events Executor are infrastructure outside the ledger's trust boundary. C-14.1
Any implementation must satisfy, at minimum:

Watches and timers - Durable watches and timers. A watch registered for a future date or C-14.2
condition fires when it is met, regardless of restarts or failures; timer state is persisted. - At-least- C-14.3
once emission. No event is ever lost; a duplicate is harmless because of idempotence checking.
- Acknowledged watches. Every declared watch is acknowledged, and the acknowledgement is C-14.4
recorded in the unit's state; a watch that remains unacknowledged beyond its arming window is
a visible, overdue item, never silence.

Delivery - Triggers carry timing, not data. A trigger says when; everything the event means C-14.5
is read from the record. - Deterministic, replayable orchestration. Each machine's own state C-14.6
is reconstructible by replaying its append-only history. - No privilege. Neither machine holds C-14.7
write access to the ledger. - Obligation liveness. Every contractual obligation is registered with C-14.8
a deadline, a test for its discharge, and a fallback action, and by its deadline it is discharged,
compensated, or declared defaulted; a duty that fails to fire is a visible, overdue item, never
silence.

14.2 The Pricing Stack and the Pricing Data Layer

- The ledger records the outputs of pricing models and never runs one. C-14.9
- Valuation is state-dependent and obtains that state from the ledger. C-14.10
- Valuations are reproducible from recorded inputs. C-14.11
- Prices and quantities are never mixed across an event. C-14.12
- Originals are preserved; adjusted values are computed; derived values are recomputed. C-14.13
- Estimates and entitlements are kept apart. C-14.14
- Model outputs re-enter only as new observations, carrying sufficient provenance. Reproduc- C-14.15
ing a valuation from recorded inputs and identified prices is unconditional; reproducing a
model's number requires the model itself, whose retention is a governance matter outside
the ledger's scope.

Scope of the Ledger Project

The Ledger project encompasses the design, formal specification, reference implementation, and C-Scope.1
verification of the following components:

- The closed double-entry move model, the conservation law, and the single-door admission C-Scope.2
function.
- The immutable, hash-chained transaction log and the projection machinery that derives all C-Scope.3
views from it.
- The three-home state architecture (ProductTerms, UnitStatus, PositionState) and the C-Scope.4
conditions that render illegal states unrepresentable.
- The smart-contract framework, its event-triggering discipline, and the purity and idempo- C-Scope.5
tence requirements.
- The market data operators and the integration layer between lifecycle state, pricing models, C-Scope.6
and the corporate-action adjustment machinery.
- The time-travel and historical-replay machinery, with its guarantees of determinism and C-Scope.7
reproducibility.

- The formal invariant catalogue and the property-based testing regime. C-Scope.8
- The settlement-projection layer. C-Scope.9
- The virtual-ledger mechanism: subordinate, non-settling instances of the same machine whose projections enter the true ledger only as stamped observations. C-Scope.10

Explicitly out of scope are: price discovery and executable pricing; legal enforceability or validity of contracts; settlement finality, payment-system access, and CSD connectivity; regulatory-submission gateways and report formatting; model governance and validation; and accounting-policy decisions. These concerns lie outside the Ledger boundary and are reconciled against it only at its perimeter. C-Scope.11

Conclusion

By making reconciliation failures structurally unreachable within its scope, by rendering broken states impossible through type discipline and admission enforcement, and by cleanly separating the admission of structurally valid transactions from the verification of their economic correctness, the Ledger framework offers a principled foundation for a post-trade system of record: simpler, because there is only one record, and more robust, because the classes of internal break that dominate conventional architectures cannot arise. Every historical state remains exactly reconstructible, and every economic outcome is traceable to a deterministic computation over recorded data.

The principal engineering challenge — and the principal opportunity — lies in the disciplined construction of the smart-contract suite that faithfully captures the terms of every instrument the system must support, and in the orchestration of the event streams that trigger those contracts. When that challenge is met, the dominant operational risk of post-trade processing — reconciliation — will have been substantially eliminated by construction rather than mitigated by process.

Authority and Amendment

This manifesto is the authoritative statement of intent and scope for the Ledger project — its architectural constitution — and the direction of authority is one-way: the manifesto is rated against nothing; everything else is rated against it. C-Auth.1 C-Auth.2 All specifications, implementations, tests, and integration layers — including every existing version of the ledger specification — shall be assessed for conformance against it and brought into line where they deviate. C-Auth.3 Where a genuine improvement requires departing from this document, the manifesto is amended explicitly first; a design choice that cannot be traced to its principles shall be rejected. C-Auth.4 Its vocabulary — unit, wallet, balance, move, transaction, watch, the immutable log, projection, the Event Monitor, the Events Executor, the Transaction Executor, smart contract, the market data operator, the three homes, virtual wallet, virtual ledger — is fixed: one name per component, and no specification shall introduce a synonym for any of them.

Amendment record

Version 1.1 (11 July 2026). Three amendments, proposed by the v15 Phase 1 Design Ruling on the collateral generalisation and ratified by the owner: Section 4's coordinate sentence restated as a signed vector — owned, lent, posted — with the five original names as its rays; Section 4 extended by one sentence fixing that non-owned coordinates carry mass never value, always under a named agreement, and that a lifecycle event extinguishes value never mass; Section 8's three-case inflow sentence sharpened to key the case on the declared legal regime of the governing agreement — settled-to-market margin as settlement, title-transfer collateral (cash included)

as financing against an obligation unit, and security-interest collateral without right of use as custody.

Version 1.2 (13 July 2026). Six changes, ratified by the owner. (i) Two commitments added — **Order** (C-2.7) and **Simulability** (C-2.8) — and Section 2 retitled accordingly; every count of the commitments now reads eight. (ii) Clause addressing embedded: every normative clause carries a margin identifier C-*sec.n* (Finding F11); the numbering is append-only — identifiers are never reused or renumbered — and the separate clause-addressed rendering is retired, this document being the sole store of the constitution's content. (iii) C-4.8 amended (ratified 2026-07-13, adopted here): every guaranteed observation enters the record only as a moveless observation-recording transaction through the one Transaction Executor — the record is the log, with no second door. (iv) C-8.7 amended (ratified 2026-07-13, adopted here): deposit-neutrality of case-2 financing is conditional on the obligation being valued at fair value, any day-one difference recognised as financing basis. (v) Section 4's coordinate clauses C-4.10–C-4.11 restated by owner's ruling, closing PARK-3: the constitution fixes the principle — ownership first and alone value-bearing; every further coordinate mass-only under a named agreement — and three admission tests, and delegates the coordinate schema to the specification on the specialists' advice. (vi) Section 12 amended by owner's ruling, closing PARK-2: C-12.1 restated — both honest answers computable at any moment, the indexing of the two views delegated to the specification — and C-12.4–C-12.5 added: repair-forward with human authorisation (views recompute automatically, money never does), and the authorised fork. No other prose changed.

Version 1.3 (14 July 2026). One change, ratified and adopted by the owner. C-4.12 added — the event universe: the kinds of event the system can process are finite, closed, and declared before any event of a kind crosses; registration names the responsible routing, so processing is total over the declared universe; an event of an undeclared kind is never processed and never lost — captured and recorded through the one door, standing as a visible open item until registration or human disposition; registry, routing, and quarantine mechanics are delegated to the specification. At adoption the owner generalised the parked wording from the undeclared-kind case to every unprocessable arrival — an undeclared kind, an unresolvable reference, a failed payload — matching the four loss paths the specification closes, and bounded the guarantee honestly at arrival: whether the world's events reach the boundary is a named trust assumption, never a fact the ledger can enforce. Closes the Event Universe Phase A constitutional question (Gap-1: routing totality nowhere required over a closed universe; Gap-2: "no event is ever lost" bound emissions only). No other prose changed.

— End of Manifesto —